

The Cost Function for Logistic Regression

The goal of a **cost function**, $J(\vec{w}, b)$, is to measure how well a set of parameters $(\vec{w}, b)$ performs on the training data. A lower cost means a better fit.

Why Not Use the Squared Error Cost?

1. The Problem of Non-Convexity:

- If we use the same squared error cost function from linear regression, $J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2$, with the logistic regression model's sigmoid function, the resulting cost function shape is **non-convex**.
- A non-convex function has many "dips" or **local minima**.
- This is a problem because **gradient descent** is not guaranteed to find the best possible parameters (the **global minimum**) and can get stuck in a suboptimal local minimum.

2. **Conclusion:** A different cost function is needed for logistic regression to ensure it is convex and that gradient descent can work reliably.

Logistic Loss Function

To build a better cost function, we first define the error on a *single* training example. This is called the **Loss Function**, denoted as $L(f_{\vec{w}, b}(\vec{x}), y)$.

The **Logistic Loss Function** is defined as follows:

- If the true label $y = 1$:
$$L(f(\vec{x}), 1) = -\log(f(\vec{x}))$$
 - If the true label $y = 0$:
$$L(f(\vec{x}), 0) = -\log(1 - f(\vec{x}))$$
-

Intuition Behind the Logistic Loss 🧠

This loss function is designed to heavily penalize predictions that are confidently wrong.

- **When the true label is 1 ($y = 1$):**
 - If the model predicts $f(\vec{x}) \rightarrow 1$ (a correct, confident prediction), the loss $-\log(1)$ approaches **0**.
 - If the model predicts $f(\vec{x}) \rightarrow 0$ (an incorrect, confident prediction), the loss $-\log(0)$ approaches **infinity**.
- **When the true label is 0 ($y = 0$):**

- If the model predicts $f(\vec{x}) \rightarrow 0$ (a correct, confident prediction), the loss $-\log(1 - 0)$ approaches **0**.
 - If the model predicts $f(\vec{x}) \rightarrow 1$ (an incorrect, confident prediction), the loss $-\log(1 - 1)$ approaches **infinity**.
-

The Full Cost Function

The overall **cost function** $J(\vec{w}, b)$ is simply the average of the loss function across all m training examples:

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\vec{w}, b}(\vec{x}^{(i)}), y^{(i)})$$

- This new cost function is **convex**, which guarantees that gradient descent can converge to the global minimum.

Simplified Cost Function for Logistic Regression

To make implementation easier, the two-part logistic loss function can be combined into a single, equivalent equation.

Simplified Loss Function

Recall the original loss function was defined in two parts, one for when $y = 1$ and another for when $y = 0$. Because y in a binary classification problem can *only* be 0 or 1, we can combine these into one expression.

- **The simplified loss function** for a single training example is:
$$L(f(\vec{x}), y) = -y \log(f(\vec{x})) - (1 - y) \log(1 - f(\vec{x}))$$
- **How it Works:**
 - **Case 1: True label is 1 ($y = 1$)**
 - The second term, $(1 - y) \log(\dots)$, becomes $0 \cdot \log(\dots) = 0$.
 - The equation simplifies to $L = -1 \cdot \log(f(\vec{x}))$, which is the original loss for $y = 1$.
 - **Case 2: True label is 0 ($y = 0$)**
 - The first term, $-y \log(\dots)$, becomes $0 \cdot \log(\dots) = 0$.
 - The equation simplifies to $L = -(1 - 0) \log(1 - f(\vec{x}))$, which is the original loss for $y = 0$.

This single-line formula is mathematically identical to the previous piecewise definition.

Full Cost Function using Simplified Loss

The overall cost function, $J(\vec{w}, b)$, is the average of this simplified loss function over all m training examples.

- **Final Cost Function for Logistic Regression:**
$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)}))]$$
- **Rationale for this Function:**
 - This cost function is derived from a statistical principle known as **Maximum Likelihood Estimation**.
 - Most importantly, it has the desirable property of being **convex**, which ensures that gradient descent can reliably find the global minimum.